

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/263087643>

Server Based Control Flocking for Aerial-Systems

Conference Paper · June 2014

DOI: 10.1109/EIT.2014.6871783

CITATIONS

8

READS

2,938

3 authors, including:



Sami Oweis

Oakland University

12 PUBLICATIONS 45 CITATIONS

SEE PROFILE



Ka Cheok

Oakland University

160 PUBLICATIONS 1,396 CITATIONS

SEE PROFILE

Server Based Control Flocking for Aerial-Systems

Sami Oweis, Subra Ganesan, and Ka C Cheok

School of Electrical and Computer Engineering

Oakland University

2200 N. Squirrel Road, Rochester, Michigan 48309-4401, USA

Email: {seweis,ganesan,cheok}@oakland.edu

Abstract—Flocking is a term that describes the behavior of a group of birds (a flock) in flight, or the swarming behavior of insects. This paper presents detailed information about how to use the flocking techniques to control a group of embedded controlled systems “Boids” such as ground systems (robotic vehicles/ swarm robots) in 2D and/or aerial systems (quadrotors) in 3D. Each one of these systems moves inside/outside of a building collectively to reach a target. This paper presents flocking behavior implemented on a server based control, which will process each of the boids properties e.g. position, speed & target. Subsequently the server will assign the appropriate move to a specific boid. The calculated information will be used locally to control and direct the movements/flocking for each boid in the group. A simulation technique and detailed flow chart is presented.

In addition to the famous Reynolds’ Rules for flocking two other rules are presented; one for targeting and another for avoiding obstacles. The targeting rule was implemented to ensure that the flock moves collectively towards a common target; it was implemented by creating a target velocity parameter. The avoiding obstacles rule was implemented to ensure that the flock doesn’t collide with obstacles that they could be in the path; it was implemented by creating an avoiding velocity parameter.

Keywords—Boids properties, flocking, rules of flocking, 2D/3D flocking, flight, constraints, robotic vehicles, swarm robots, quadrotors and path planning.

I. INTRODUCTION

Flocking is a behavior that shows a set of independent self-propelled entities moving together collectively [1]. To participate in a flock, each entity must be able to coordinate its own movements with that of its neighbor entities. An entity cannot move too fast or too slow relative to the flock; it also cannot be too close or too far from any of its closest neighbor entities. Each entity must continuously adjust its motion, so not to disrupt any other neighbor entity. Flocking behavior can be found both in natural and artificial environments. Common natural flocks include bird formations or schools of fish. Artificial flocking behavior can often be found in robotics and aerospace.

II. ALGORITHMIC COMPLEXITY

In a normal flocking behavior, there is no central control; each bird behaves autonomously. In other words, each bird has to decide for itself which flocks to consider as its environment. Usually, environment is defined as a circle (2D) or an orb (3D) with a certain radius, representing reach [2].

A basic implementation of a flocking algorithm has a time complexity of $O(n^2)$ [2] - each bird searches through all

other birds to find those who fall into this environment, where n is the number of the birds in the flock. Throughout the algorithm applied, this paper presents a new way of flocking behavior, Server Based Control Flocking Algorithm which it can be implemented on a base processing server; that being implemented to have an vision overview and control of each boid in the flock.

III. BOIDS PROPERTIES

There are two sets of properties for the boids: global and environmental parameters, such as radius, coefficient of static and kinetic friction, and acceleration. There are other local boids parameters, such as mass, separation, alignment, cohesion, targeting and avoiding strengths, velocity and force limits, in addition to the number ID to uniquely identify each boid in the space.

IV. REYNOLDS RULES OF FLOCKING

Flocking behavior in a virtual environment was first demonstrated by Craig Reynolds, in 1986. Rather than scripting a path that each entity (coined boids) would follow, Reynolds developed a set of simple adaptive rules that each boid could apply to its current position, speed, and orientation. These rules are known as Reynolds Rules of Flocking. There are three main rules: separation, alignment, and cohesion. As shown in figure 1.a, figure 1.b, and figure 1.c respectively.

A. Flocking Rule 1 Separation

This rule ensures that each boid does not get too close to any boid near it. This rule is necessary to prevent crowding. It is being applied by creating a velocity parameter, by using the following equation:

$$V_{si}(k+1)_{x,y,z} = \left[s^* \sum_{n \in N} \left(\frac{1}{Dx_{i,n}(k)}, \frac{1}{Dy_{i,n}(k)}, \frac{1}{Dz_{i,n}(k)} \right) \right] \quad (1)$$

Where:

$V_{si}(k+1) \rightarrow$ The velocity of the current boid.

$N \rightarrow$ The number of the boids counted within the neighborhood.

$Dx_{i,n}(k) = P_{ix}(k) - P_{nx}(k)$ The distance in x direction.

$Dy_{i,n}(k) = P_{iy}(k) - P_{ny}(k)$ The distance in y direction.

$Dz_{i,n}(k) = P_{iz}(k) - P_{nz}(k)$ The distance in z direction.

$P_i(k) \neq P_n(k)$ for all $n \in N$.

$P_i(k) \rightarrow$ The position of the current boid.

$P_n(k) \rightarrow$ The position of a neighbor from the set N .

$s \rightarrow$ The separation strength.

B. Flocking Rule 2 Alignment

This rule ensures that each boid generally matches the velocities of boids near it. This rule is necessary to ensure that the flock moves collectively in a common direction. It is being applied by creating an alignment velocity parameter, by using the following equation:

$$V_{ai}(k+1) = \left[a * \left(\frac{\sum_{n \in N} V_n(k)}{N} \right) \right] \quad (2)$$

Where:

$V_{ai}(k+1) \rightarrow$ The alignment velocity of the current boid.

$V_n(k) \rightarrow$ The velocity of the neighbor boid from the Set N .

$a \rightarrow$ The alignment strength.

C. Flocking Rule 3 Cohesion

This rule ensures that each boid steers to move towards the average position of its neighbor boids. This rule ensures that the boids do not break away from the flock. It is being applied by creating a cohesion velocity parameter, by using the following equation:

$$V_{ci}(k+1) = \left[c * \left(\frac{\sum_{n \in N} P_n(k)}{N} \right) - P_i(k) \right] \quad (3)$$

Where:

$V_{ci}(k+1) \rightarrow$ The cohesion velocity of the current boid.

$P_i(k) \rightarrow [P_{ix}(k), P_{iy}(k), P_{iz}(k)]$ The position of the current boid.

$P_n(k) \rightarrow [P_{nx}(k), P_{ny}(k), P_{nz}(k)]$ The position of a neighbor from the Set N .

$c \rightarrow$ The cohesion strength.

Figure 2 below shows an example of a group of boids are heading toward a specified target, while the boids travel path was super imposed to keep track of it for demonstration.

V. ADDITIONAL FLOCKING RULES

While not part of the original Reynolds Rules of Flocking, two additional rules have been implemented for a more interesting flocking behavior.

A. Flocking Rule 4 Targeting

This rule forces each boid steers to move towards a target point in space. This rule is necessary to ensure that the flock moves collectively towards a common target. It is being applied by creating a target velocity parameter, by using the following equation:

$$V_{ti}(k+1) = \left[t * (P_T - P_i(k)) \right] \quad (4)$$

Where:

$V_{ti}(k+1) \rightarrow$ The targeting velocity of the current boid.

$P_i(k) \rightarrow [P_{ix}(k), P_{iy}(k), P_{iz}(k)]$ The position of the current boid.

$P_T \rightarrow [P_{Tx}, P_{Ty}, P_{Tz}]$ The position of the target

$t \rightarrow$ The targeting strength.

B. Flocking Rule 5 Avoidance

This rule forces each boid to avoid a particular point in space. It is essentially a global repulsive force, similar to the separation rule. It is being applied by creating an avoiding velocity parameter, by using the following equation:

$$V_{ri}(k+1) = \left[r * G * (sign(P_i(k)) - sign(P_R)) \right] \quad (5)$$

Where $G \Rightarrow$ is the obstacle avoidance function and its defined in the below equation

$$G = \left\{ \frac{1}{\sqrt{(P_{ix}(k) - P_{Rx})^2 + (P_{iy}(k) - P_{Ry})^2 + (P_{iz}(k) - P_{Rz})^2}} \right\}$$

Where:

$V_{ri}(k+1) \rightarrow$ The avoidance velocity of the current boid.

$P_i(k) \rightarrow [P_{ix}(k), P_{iy}(k), P_{iz}(k)]$ The position of the current boid.

$P_R \rightarrow [P_{Rx}, P_{Ry}, P_{Rz}]$ The position of the obstacle to be avoided.

$r \rightarrow$ The avoiding strength.

$sign(P_{i,R}) \rightarrow [sign(P_x), sign(P_y), sign(P_z)]_{i,R}$.

C. Application with Newtonian Physics

In order to get realistic flocking behavior in a virtual environment, special care must be taken to ensure that a proper physics model is used. Factors such as z-acceleration (gravity), friction, mass, velocity, and force limits must be incorporated. First, the resultant velocity parameter was calculated for each boid, which takes into account all the flocking rules and velocity limit. The velocity limit is necessary to make sure the boids do not accelerate too fast.

$$V = \left[V_{si}(k+1) + V_{ai}(k+1) + V_{ci}(k+1) + V_{ti}(k+1) + V_{ri}(k+1) \right] \quad (6)$$

Where:

$\|V\| \leq V_{limit}$

$V \rightarrow$ The resultant velocity of the current boid.

Using this resultant velocity, the acceleration parameter was calculated for the current boid.

$$A_i(k+1) = \left[V - V_i(k+1) \right] \quad (7)$$

Where:

$A_i(k+1) \rightarrow [A_{ix}, A_{iy}, A_{iz}]$ The acceleration of the current boid.

Using the acceleration parameter, the force parameter can be calculated for the current boids applied force, using Newtons 2nd law.

$$F_a = \left[mA_i(k+1) \right] \quad (8)$$

Where:

$\|F_a\| \leq F_{limit}$

$F_a \rightarrow [F_{ax}, F_{ay}, F_{az}]; F \rightarrow [F_{limit,x}, F_{limit,y}, F_{limit,z}]$

Friction is the force that resists the relative motion of two

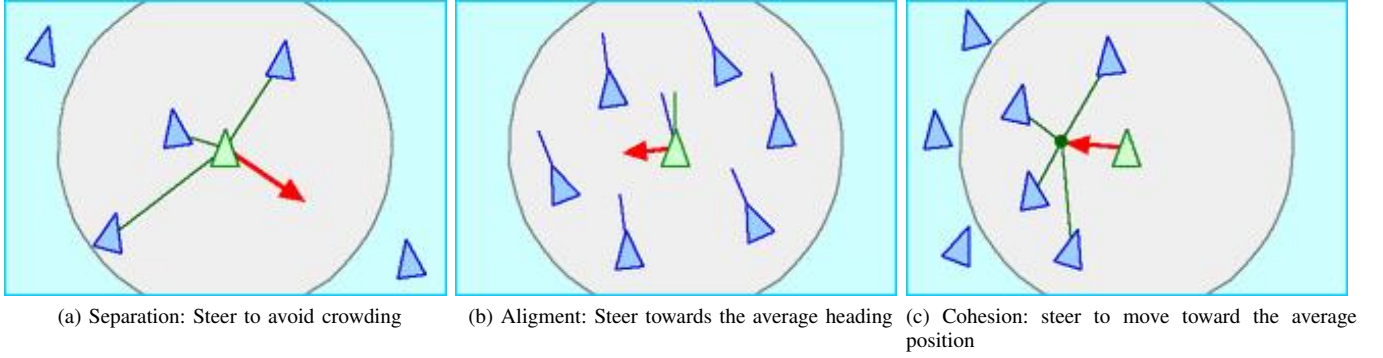


Fig. 1: Reynolds Rules of Flocking

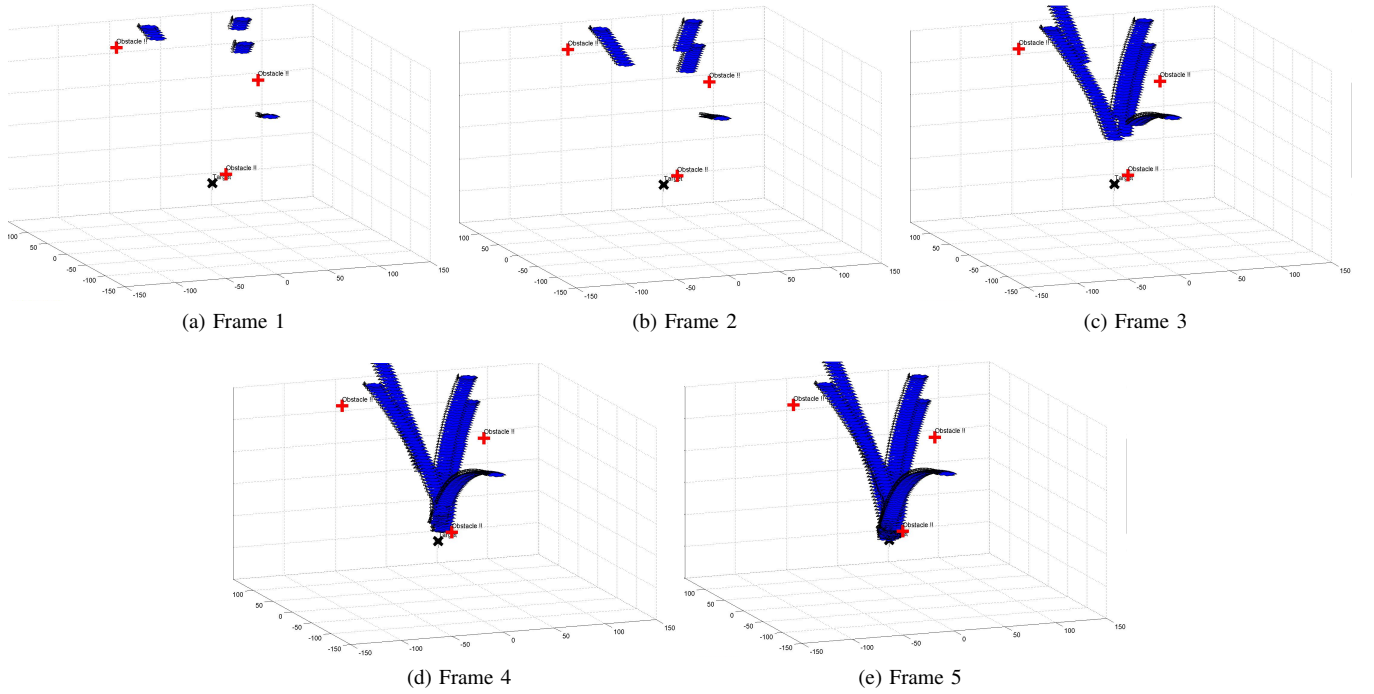


Fig. 2: Boids (blue) are heading toward the target (black), obstacles (Red). Frame @5x faster, Space dimensions (2D or 3D)

surfaces in contact with each other. The standard model for friction is divided into static friction and kinetic friction. Static friction prevents an object to move if an applied force cannot overcome it. Kinetic friction applies a negative force relative to the applied force, typically slowing down the object.

$$F_f(k) = \begin{cases} [-F_i(k)] & , \text{if } |F_i(k)| \leq \mu_s mg \\ [-\text{sign}(F_i(k)) \mu_k mg] & , \text{if } |F_i(k)| > \mu_s mg \end{cases} \quad (9)$$

Where

$$\text{sign}(F_i) \rightarrow [\text{sign}(F_{ix}), \text{sign}(F_{iy}), \text{sign}(F_{iz})].$$

Once the friction force is known, the resultant force parameter can be calculated.

$$F_f(k+1) = [F_i(k) + F_a + F_f(k)] \quad (10)$$

Because the resultant force parameter takes into account the cumulative forces applied to the boid (or its momentum), the boids new velocity can be calculated as follows:

$$V_i(k+1) = \left[\frac{F_i(k+1)}{m} \right] \quad (11)$$

With the next velocity parameter known, the next position of the current boid can be determined.

$$P_i(k+1) = [P_i(k) + V_i(k+1)] \quad (12)$$

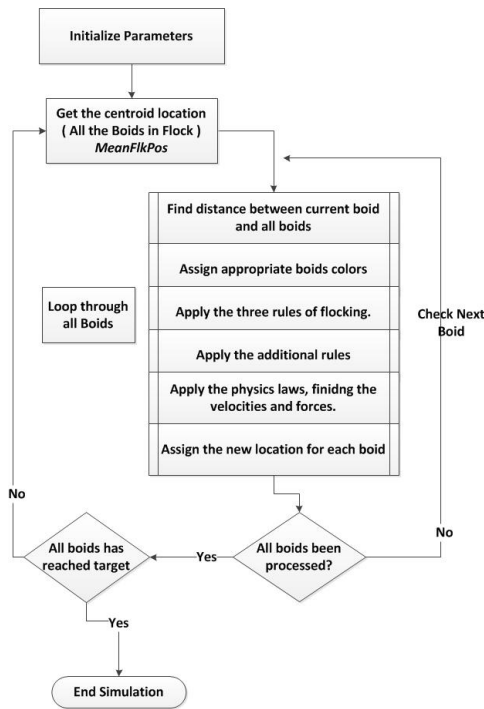


Fig. 4: The Flocking Proces Flow Chart

VI. IMPEMNTATION

A. Simulation

The Matlab software [6] was used to build a Graphical User Interface (GUI) shown in Figure 3. The GUI allows the user to change on-fly the main input/signal parameters in both types; global and local parameters.

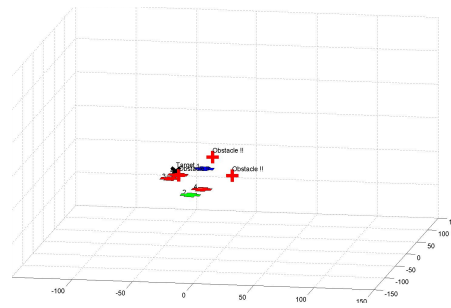
Figure 5 below shows an example of a group of boids identifying an obstacle in thier path toward a specified target in space. Boids travel location was super imposed to track it.

The GUI contains the following:

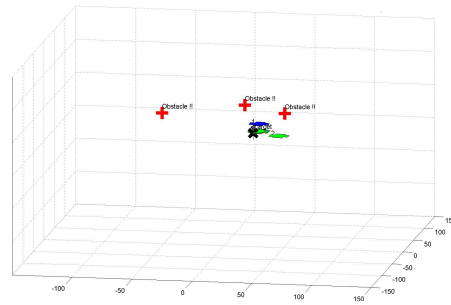
- Simulation space selection, It allows the user to select weather 2D or 3D is desired.
- Global and boid parameters.
- Graphical visualization, which shows the simulation output.
- Target is presented in a *black* color objects.
- Obstacle is presented in a *red* color objects.
- Boids are presented in three colors:
 - *Blue* color objects as they are not part of the flock yet.
 - When a boid is getting too close to the obstacle, its turns to *red* color. That is shown in the figure 5.
 - When two or more boids are in cohesion with each others, they turn into *yellow* color. That is shown in the figure 6.

B. The Simulation Process

- First Initializing the GUI interface.
 Second The user will update the parameters as suitable to the problem.



(a) Cohesion 1



(b) Cohesion 2

Fig. 6: Flocking simulation shows boids (yellow) in cohesion

Third Initializing the global and Boids properties based on the parameters used.

Run While simulation is enabled ...

- 1) Randomization for both obstacles and target locations, - Keeping a timer for randomly changing the location of the obstacle, for dynamic object avoiding.
- 2) Calculating the centroid of the flock (mean of the difference between the flock boids locations).
- 3) The centroid is being compared to the target while monitoring the offset between the boids locations and the location(s) of the obstacle(s).
- 4) While the boids position, velocity, friction forces, and other properties are calculated for the current position, the new position gets assigned.
- 5) Initializing velocity separation, alignment, and cohesion parameters.
- 6) Loop for each boid:
 - a) Get the distance between the current boid and the rest of the flock.
 - b) If close by boid where found within limits - Increment the "Neighbors" number. - Identify the boids with other color.
 - c) Apply the Reynolds rules calculations, *equations(1-3)*
 - d) Apply the additional rules calculations, *equations(4-7)*
 - e) Calculate current boid's applied and friction force vectors, *equations(8-10)*
 - f) Calculate current boid's resultant force vector, *equation(11)* - Update the simulated boids with the new positions and properties, *equation(12)*

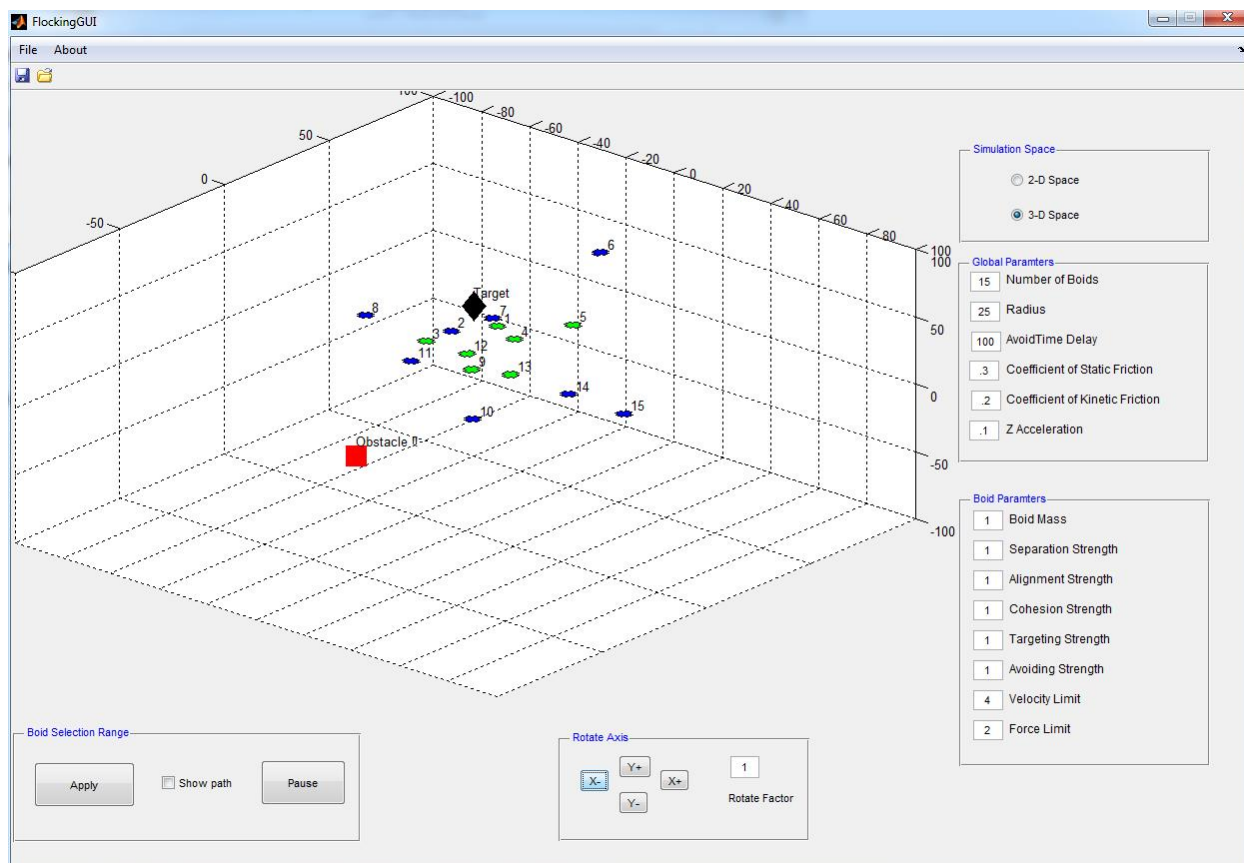


Fig. 3: Flocking Simulation GUI

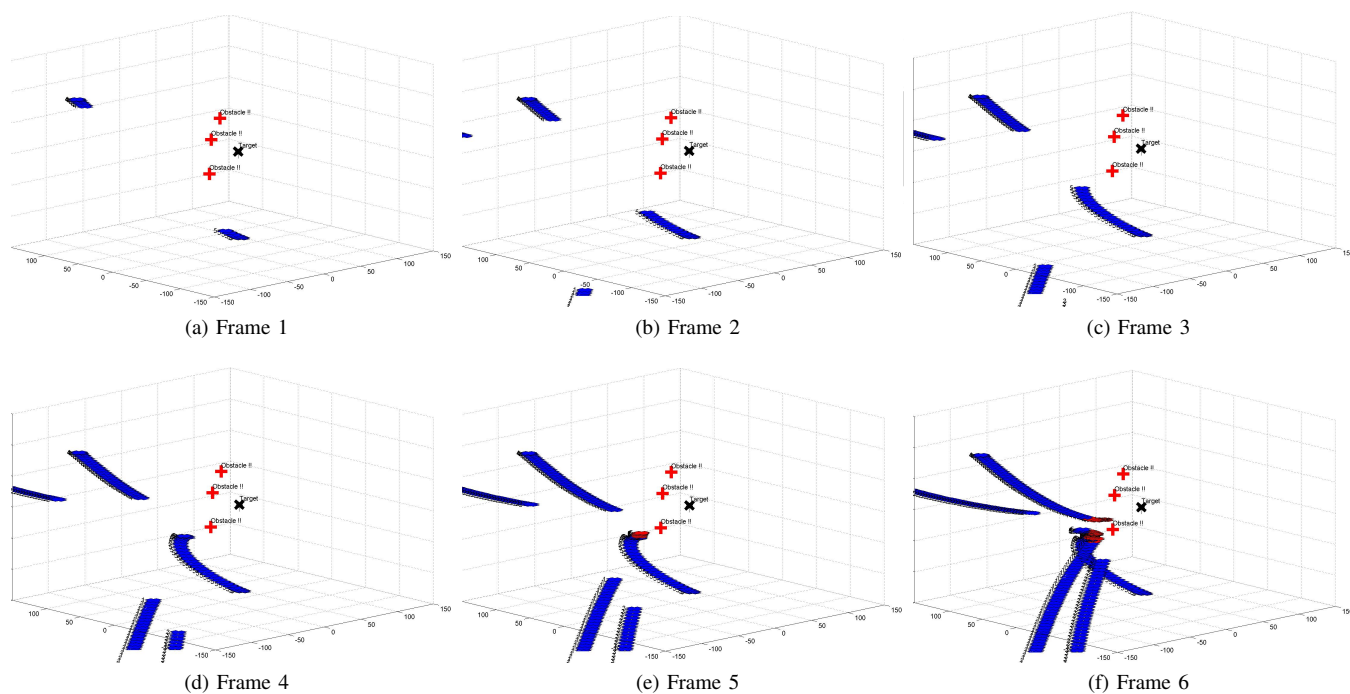


Fig. 5: Boids (blue) Identifying an obstacles (red) in the path heading to the target (black). Frame @5x faster

VII. CONCLUSION

In this paper we described application of two new rules, i.e. targeting and avoiding for flocking control using centralized server. We described the simulation in detail and with flowchart. The simulation used in this paper, shows that "Flocking Algorithm" can be implemented on a server base, that will calculate and determine the boids' properties, such as, velocity, position, cohesion, and targeting information. The calculated information will be used locally to control and direct the movements/flocking of a group of boids.

C. W. Reynolds added in his paper [1] "Sparrows might flock around a group of obstacles that is in fact a herd of elephants. Similarly, behavioral obstacles might not merely be in the way; they might be objects of fear such as predators. It has been noted that natural flocking instincts seem to be sharpened by predators.

This work may be applied to many problems like, running away from heat sources in a case of a fire, or natural gas leak source. Another could be in terms of birds getting closer to a school of fish, and alligators are in a reaching distance. Future work for this paper will be to include an optimization matrices calculation to bring the process time lower, more improved- on conditional rules. Another future work is to compare the current proposed methods, to, Fuzzy Logic (FL), Genetic Algorithm (GA), or Neural Network (NN) methods.

ACKNOWLEDGMENT

The authors would like to thank both of Mr. Suat Kuesfoglul and Mr. Dariusz Mikulski for their initial cooperation on a flocking simulation we have done together.

REFERENCES

- [1] <http://www.michaelshell.org/tex/ieeetran/>
- [2] C. W. Reynolds, "Flocks, Herds, and Schools: A Distributed Behavioral Model," in ACM SIGGRAPH '87, Anaheim, California, 1987
- [3] "Flocking Behavior," [Online]. Available: [http://en.wikipedia.org/wiki/Flocking_\(behavior\)](http://en.wikipedia.org/wiki/Flocking_(behavior)).
- [4] "Boids," [Online]. Available: <http://www.red3d.com/cwr/boids/>.
- [5] D. M. B. a. G. Seemann, "Chapter4_Flocking," in AI for Game Developers, Creating Intelligent Behavior in Games.
- [6] "Classical Mechanics," [Online]. Available: http://en.wikipedia.org/wiki/Classical_mechanics.
- [7] "MATLAB," [Online]. Available: www.Mathwork.com.
- [8] Ibanez J, Gomez-Skarmeta A F, Blat J (2003). "DJ-boids: emergent collective behavior as multichannel radio station programming". Proceedings of the 8th international conference on Intelligent User Interfaces. pp. 248250. doi:10.1145/604045.604089.
- [9] Swarm behaviour model by University of Groningen
- [10] Spector, L.; Klein, J.; Perry, C.; and Feinstein, M. (2003). "Emergence of Collective Behavior in Evolving Populations of Flying Agents". Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2003). Springer-Verlag. Retrieved 2007-05-01.
- [11] "http://psychcentral.com/news/2008/02/15/herd-mentality-explained/1922.html". Retrieved on October 31st 2008.
- [12] Moere A V (2004). "Time-Varying Data Visualization Using Information Flocking Boids". Proceedings of the IEEE Symposium on Information Visualization. pp. 97104. doi:10.1109/INFVIS.2004.65.
- [13] Cui Z, Shi Z (2009). "Boid particle swarm optimisation", International Journal of Innovative Computing and Applications 2 (2): 7785. doi:10.1504/IJICA.2009.031778.
- [14] Gabbai, J. M. E. (2005). Complexity and the Aerospace Industry: Understanding Emergence by Relating Structure to Performance using Multi-Agent Systems. Manchester: University of Manchester Doctoral Thesis.
- [15] Ibanez J, Gomez-Skarmeta A F, Blat J (2003). "DJ-boids: emergent collective behavior as multichannel radio station programming". Proceedings of the 8th international conference on Intelligent User Interfaces. pp. 248250. doi:10.1145/604045.604089.



Sami R. Oweis was born in Zarka, Jordan, he received his Bachelor of Science in Engineering (B.S.E) degree in Computer Engineering, in 2007 and the Masters in Science (M.S.) degree in electrical and computer engineering, in 2008 from Oakland University, Rochester, MI, 48309, USA. He is currently pursuing his Ph.D in artificial intelligent and controls engineering from Oakland University. He has been working as a teaching and research assistant for the electrical and computer engineering department at Oakland University during this time. He has taught and helped students in the areas of digital signal and imaging processing, microcontrollers, micro based computer systems, embedded application, artificial intelligent and control theory as well as using MATLAB/Simulink